

Automating Tactically and Strategically

FiSTB TESTING ASSEMBLY 19-20.9.2017, Helsinki

Alan Richardson

- www.eviltester.com
- www.compendiumdev.co.uk
- [@eviltester](https://twitter.com/eviltester)

Synopsis

Do you automate tactically or strategically?

Both approaches are useful and can help our test projects in different ways.

Synopsis

Thinking about automating tactically allows us to move fast, target immediate needs, support testing and gain benefits very quickly - often with minimal cost and justification.

Automating strategically allows us to gain the backing for longer periods of work and justify the cost and ongoing maintenance.

Synopsis

If we believe we are automating strategically but we actually automate tactically then we may create problems for ourselves: we may not have the time for maintenance and refactoring, we may not use our 'automation' to support our 'manual testing'.

Synopsis

Other dangers and problems exist, which we will learn about in this keynote when we will compare the two approaches, learn the dangers and the benefits, and learn how to combine tactical and strategic approaches to better our testing efforts.

Secrets of Successful Practical Automating

- Get work done
- Automate Flows
- Multiple Abstractions
- Abstract to libraries, not frameworks

Testing field argues about

- Testing vs Checking
- You can't automate testing

"Test Automation" is contentious

An Example "This is a test?"

```
@Test
public void createPreviewMVP() throws IOException {

    String args[] = {
        new File(System.getProperty("user.dir"),
            "pandocifier.properties").getAbsolutePath()};

    new PandocifierCLI().main(args);

    String propertyFileName = args[0];
    File propertyFile = new File(propertyFileName);
    PandocifierConfig config;
    config = new PandocifierConfigFileReader().
        fromPropertyFile(propertyFile);

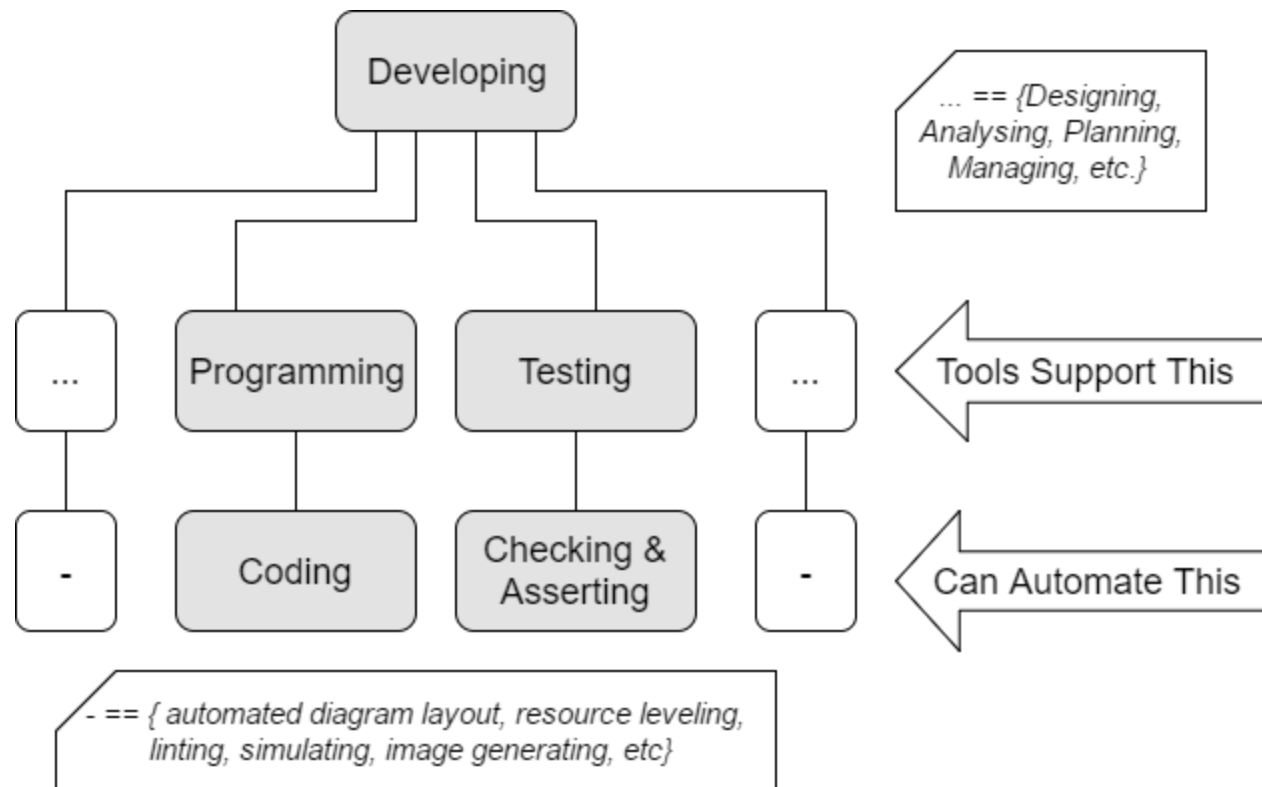
    Assert.assertTrue(new File(
        config.getPreviewFileName()).exists());
}
```


An Example "Ce n'est pas un test"

- This is a Test because it says it is a `@Test`
- This is an Automated Test because it **checks** something (file exists)
- This is an Automated Test because it **Asserts** on the check
- This is a Tool - I use it to automatically build one of my books
- This is not an Application - it doesn't have a GUI or a standalone execution build

This is **automated execution of a process**

We Automate Parts of our Development Process: Coding, Checking, Asserting, ...



Tactical Reality - I don't really care, I just want to get work done

Suggests we also have 'strategic reality'

Tools vs Abstractions

- Gherkin
 - Specflow, Cucumber
-

- Gherkin is a DSL templating language
- Specflow, Cucumber are template parsers
- We have to create abstractions to fill in the gap

Abstraction

"The purpose of abstraction is not to be vague, but to create a new semantic level in which one can be absolutely precise."

- *E. W. Dijkstra*
- 1972 ACM Turing Lecture: '[The Humble Programmer](#)'

Regression Testing

The system rarely actually suffers "Regression"

- TDD, ATDD, Automated execution
- All of these provide an ongoing safety net which might mean we don't need to revisit areas in the future (if we also tested them)
- They don't mean that we have 'tested' them now

Coverage

- All coverage is model based
- Code coverage based on source code model
- What about requirements, system interfaces, states, flows?
- Code coverage is a useful metric for TDD coverage
 - coverage models based on risk

Tactical vs Strategic Automating

- Strategic does not mean 'strategy document'

Automating as part of a Test Strategy

Automating Tactically vs Strategically

What is Tactical?	What is Strategic?
Solve a problem	Agreed & Understood Goals
Short Term	Long Term
Change as necessary	Slower to Change
Your Time	'project' time

Warning - sometimes tactics look like strategy

- Use of Jenkins is not Continuous Integration
- Cucumber/Specflow (Gherkin) != ATDD or BDD
- Automated Deployments are not Continuous Delivery
- Containers are not always good environment provision
- Page Objects are not guaranteed good system abstractions

You know you are Automating strategically when...

- Reduced Maintenance
- Reduced change
- Everyone agrees why
- No fighting for 'time'
- It's not about roles
- It gets done

How do we automate?

- not about "test automation"
- it is about automating tasks
- automate the assertions used to check that the stories are still 'done'
- Automate
 - flows through the system
 - vary data
 - abstract the execution

Automating Strategically

Abstractions - "modeling the system in code"

- abstractions are a big part of automating strategically
- abstractions are about modelling - people seem to be scared of having too many models

see also "Domain Driven Design"

No Abstractions - Web Testing

```
@Test
public void canCreateAToDoWithNoAbstraction(){
    driver = new FirefoxDriver();
    driver.get(
        "http://todomvc.com/architecture-examples/backbone/");

    int originalNumberOfTodos = driver.findElements(
        By.cssSelector("ul#todo-list li")).size();

    WebElement createTodo;
    createTodo = driver.findElement(By.id("new-todo"));
    createTodo.click();
    createTodo.sendKeys("new task");
    createTodo.sendKeys(Keys.ENTER);
    assertTrue(driver.findElement(
        By.id("filters")).isDisplayed(), is(true));

    int newTodos = driver.findElements(
        By.cssSelector("ul#todo-list li")).size();
    assertTrue(newTodos, greaterThan(originalNumberOfTodos));
}
```

But there were Abstractions

- WebDriver - abstraction of browser
- FirefoxDriver - abstraction Firefox (e.g. no version)
- WebElement - abstraction of a dom element
- createToDo - variable - named for clarity
- Locator Abstractions via methods `findElement` `findElements`
- Manipulation Abstractions `.sendKeys`
- Locator Strategies `By.id` (did not have to write CSS Selectors)
- ...

Lots of Abstractions

Using Abstractions - Same flow

```
@Test
public void canCreateAToDoWithAbstraction(){
    TodoMVCUser user =
        new TodoMVCUser(driver, new TodoMVCSite());

    user.opensApplication().and().createNewToDo("new task");

    ApplicationPageFunctional page =
        new ApplicationPageFunctional(driver,
                                      new TodoMVCSite());

    assertThat(page.getCountOfToDoItems(), is(1));
    assertThat(page.isFooterVisible(), is(true));
}
```

What Abstractions were there?

- `TodoMVCUser`
 - User
 - Intent/Action Based
 - High Level
- `ApplicationPageFunctional`
 - Structural
 - Models the rendering of the application page

REST Test - No Abstractions

```
@Test
public void aUserCanAccessWithBasicAuthHeader(){

    given().
        contentType("text/xml").
        auth().preemptive().basic(
            TestEnvDefaults.getAdminUserName(),
            TestEnvDefaults.getAdminUserPassword()).
    expect().
        statusCode(200).
    when().
        get(TestEnvDefaults.getURL().toExternalForm() +
            TracksApiEndpoints.todos);
}
```

But there were Abstractions

- RESTAssured - `given`, `when`, `then`, `expect`, `auth`, etc.
- RESTAssured is an abstraction layer
- Environment abstractions i.e. `TestEnvDefaults`

Lots of Abstractions

But a lack of flexibility

Different abstractions to support flexibility

```
@Test
public void aUserCanAuthenticateAndUseAPIWithBasicAuth(){

    HttpMessageSender http = new HttpMessageSender(
        TestEnvDefaults.getURL());

    http.basicAuth( TestEnvDefaults.getAdminUserName(),
        TestEnvDefaults.getAdminUserPassword());

    Response response = http.getResponseFrom(
        TracksApiEndpoints.todos);

    Assert.assertEquals(200, response.getStatusCode());
}
```

An API Abstraction

```
@Test
public void aUserCanAuthenticateAndUseAPIWithBasicAuth(){

    TodosApi api = new TodosApi(
        TestTestEnvDefaults.getURL(),
        TestEnvDefaults.getUser());

    Response response = api.getTodos();

    Assert.assertEquals(200, response.getStatusCode());
}
```

Automating API vs GUI

Automate at the interfaces where you can interact with the system.

Architecting to Support...

- Testing
- Deployment
- Monitoring
- Releasing
- Support
- Automating
- Devops

Abstractions support different types of testing

- regular automated execution with assertions
- creation and maintenance by many roles
- exploratory testing
- stress/performance testing (bot example) requires thread safe abstractions

Good abstractions encourage creativity, they do not force compliance and restrict flexibility.

Supports Law of Requisite Variety

"only variety can destroy variety"

- W. Ross Ashby

"only variety can absorb variety"

- Stafford Beer

Frameworks can 'Destroy' variety

- Frameworks force us to comply with their structure
 - or things get 'hard'
 - or we don't reap the benefits from the Framework
- Some things are not possible
- Some things never occur to you to do

Models

We have a lot of models

- user intent model - what I want to achieve
- user action model - how I do that
- interface messaging model - how the system implements that
- admin models, support models, GUI models

And models overlap

- user can use the API or the GUI to do the same things

When we don't do this

- test code takes a long time to maintain
- test code takes too long to write
- test code is hard to understand
- test code is brittle - "every time we change the app the tests break/ we have to change test code"
- execution is flaky - intermittent test runs - "try running the tests again"

Example: Tactical Approach on Agile projects

- exploratory testing to get the story to done
- create a tech debt backlog for 'missing' automated tests
- `@Test` code is tactical and we can knock it up (not as good as production)

Example: Strategic Approach on Agile Projects

- automate assertion checking of acceptance criteria for "done"
- ongoing execution of the assertions for the life of the story
- implies
 - maintain this for the life of the story in the application
 - deleting the @Test code when no longer relevant
 - amending code structure to make relevant as stories change and evolve
 - @Test code is strategic asset (just like production code)

Why labour the point?

- "test automation" often means "testers do the automating"
- testers may not program as well as programmers
- testers may not know various patterns of writing code
- when programmers see `@Test` code written by testers they often don't want to touch it or maintain it

Move from Abstractions such as 'Programming' and 'Testing' to *'Developing'*

I say I'm a "test consultant" but the reality is that I'm a "software development consultant", and part of the process of developing is testing.

Summary

- Automate Strategically on Projects
- ongoing assertion checking automated in @Test code
- Abstractions model the system in code
- Write abstractions at the domain level of the assertion concern
- Good abstractions can be used to support exploratory testing
- Write good code all the time

Learn to "Be Evil"

- www.eviltester.com
- [@eviltester](https://twitter.com/eviltester)
- www.youtube.com/user/EviltesterVideos

Learn About Alan Richardson

- www.compendiumdev.co.uk
- uk.linkedin.com/in/eviltester

Follow

- LinkedIn - [@eviltester](#)
- Twitter - [@eviltester](#)
- Instagram - [@eviltester](#)
- Facebook - [@eviltester](#)
- Youtube - [EvilTesterVideos](#)
- Pinterest - [@eviltester](#)
- Github - [@eviltester](#)
- Slideshare - [@eviltester](#)

BIO

Alan is a test consultant who enjoys testing at a technical level using techniques from psychotherapy and computer science. In his spare time Alan is currently programming a [multi-user text adventure game](#) and some [buggy JavaScript games](#) in the style of the Cascade Cassette 50. Alan is the author of the books "[Dear Evil Tester](#)", "[Java For Testers](#)" and "[Automating and Testing a REST API](#)". Alan's main website is compendiumdev.co.uk and he blogs at blog.eviltester.com